

BỘ MÔN KHOA HỌC DỮ LIỆU

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

LAB 05 – ARTIFICIAL INTELLIGENCE



TS. TRẦN NGỌC VIỆT



HỌC KỲ 1 – NĂM HỌC 2024-2025



KHÓA K28 & K29

#Thực hành 01:

#Bài toán tìm cây khung nhỏ nhất trong đồ thị

Cây khung nhỏ nhất, liên quan đến cây và liệt kê dưới đây vì đây là những kiến thức rất thường gặp trong những bài tập về cây khung

Cây khung nhỏ nhất là gì

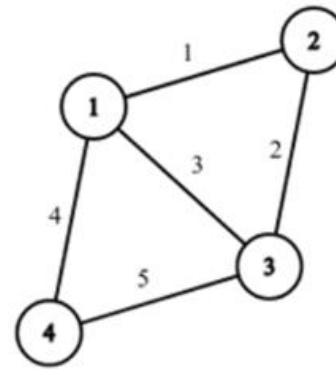
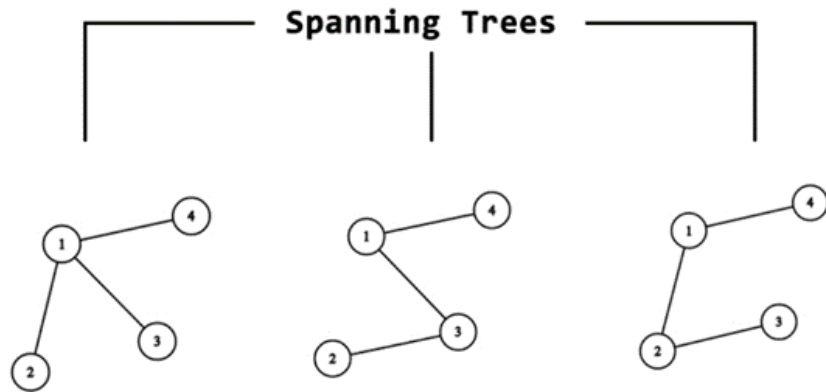
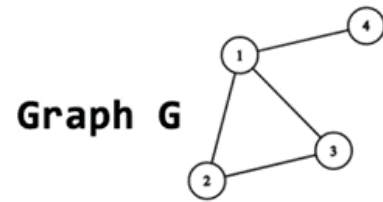
Biết rằng 1 đồ thị được biểu diễn bằng công thức $G=(V, E)$ trong đó đồ thị G của chúng ta bao gồm tập các đỉnh V và tập các cạnh E .

+**Cây khung (*spanning tree*)** của đồ thị là một tập hợp các cạnh của đồ thị thỏa mãn tập cạnh này ***không chứa chu trình*** và ***liên thông*** (từ một đỉnh bất kì có thể đi tới bất kỳ đỉnh nào khác theo mà chỉ dùng các cạnh trên **cây khung**)

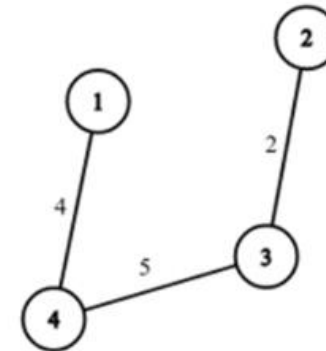
+Trong đồ thị **có trọng số**, **cây khung nhỏ nhất (*minimum spanning tree*)** là **cây khung** có tổng trọng số các cạnh trong cây **nhỏ nhất**.

#Thực hành 01:

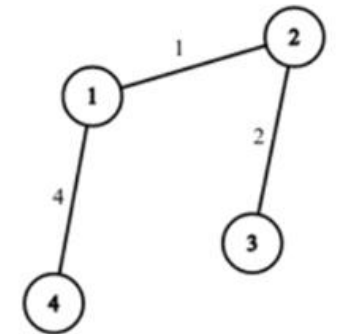
+Một ví dụ về cây khung trong đồ thị vô hướng không trọng số



Undirected
Graph



Spanning
Tree
Cost = 11(=4+5+2)



Minimum Spanning
Tree
Cost = 7(=4+1+2)

#Thực hành 01:

Kruskal's algorithm

Cây khung nhỏ nhất từ 1 đỉnh

đồ thị vô hướng, có trọng số

class Graph:

```
def __init__(self, vertices):  
    self.V = vertices  
    self.graph = []
```

add cạnh vào đồ thị

```
def addEdge(self, u, v, w):  
    self.graph.append([u, v, w])
```

tìm tập hợp thứ i

```
def find(self, parent, i):  
    if parent[i] != i:  
  
        parent[i] = self.find(parent, parent[i])  
    return parent[i]
```

|
def union(self, parent, rank, x, y):

```
    if rank[x] < rank[y]:  
        parent[x] = y  
    elif rank[x] > rank[y]:  
        parent[y] = x
```

else:

```
    parent[y] = x  
    rank[x] += 1
```

#Thực hành 01:

```
# ctrình chính cấu trúc MST
# Kruskal's algorithm
def KruskalMST(self):

    # tập lưu KetQua MST
    result = []
    # biến chỉ mục dùng sắp xếp cạnh
    i = 0

    # biến chỉ mục, dùng kết quả
    e = 0
    self.graph = sorted(self.graph, key=lambda item: item[2])

    parent = []
    rank = []

    for node in range(self.V):
        parent.append(node)
        rank.append(0)

    while e < self.V - 1:

        u, v, w = self.graph[i]
        i = i + 1
        x = self.find(parent, u)
        y = self.find(parent, v)

        if x != y:
            e = e + 1
            result.append([u, v, w])
            self.union(parent, rank, x, y)
```

#Thực hành 01:

```
minimumCost = 0
print("Edges in the constructed MST")
for u, v, weight in result:
    minimumCost += weight
    print("%d -- %d == %d" % (u, v, weight))
print("Minimum Spanning Tree", minimumCost)
```

```
if __name__ == '__main__':
```

```
    g = Graph(4)
```

```
    g.addEdge(0, 1, 10)
```

```
    g.addEdge(0, 2, 6)
```

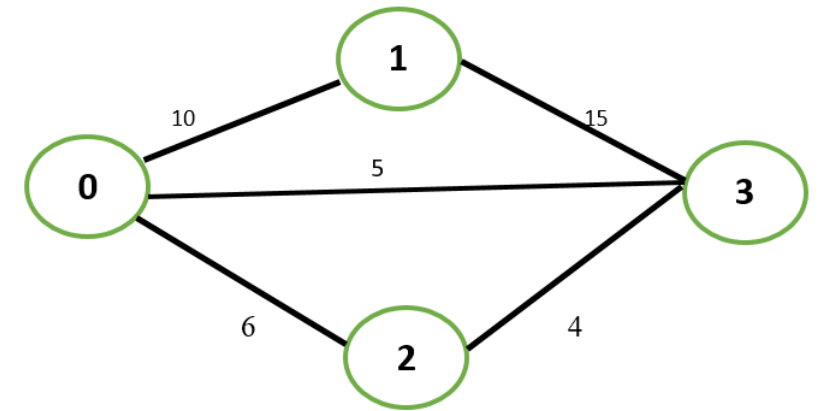
```
    g.addEdge(0, 3, 5)
```

```
    g.addEdge(1, 3, 15)
```

```
    g.addEdge(2, 3, 4)
```

```
    g.KruskalMST()
```

#đồ thị vô hướng, có trọng số

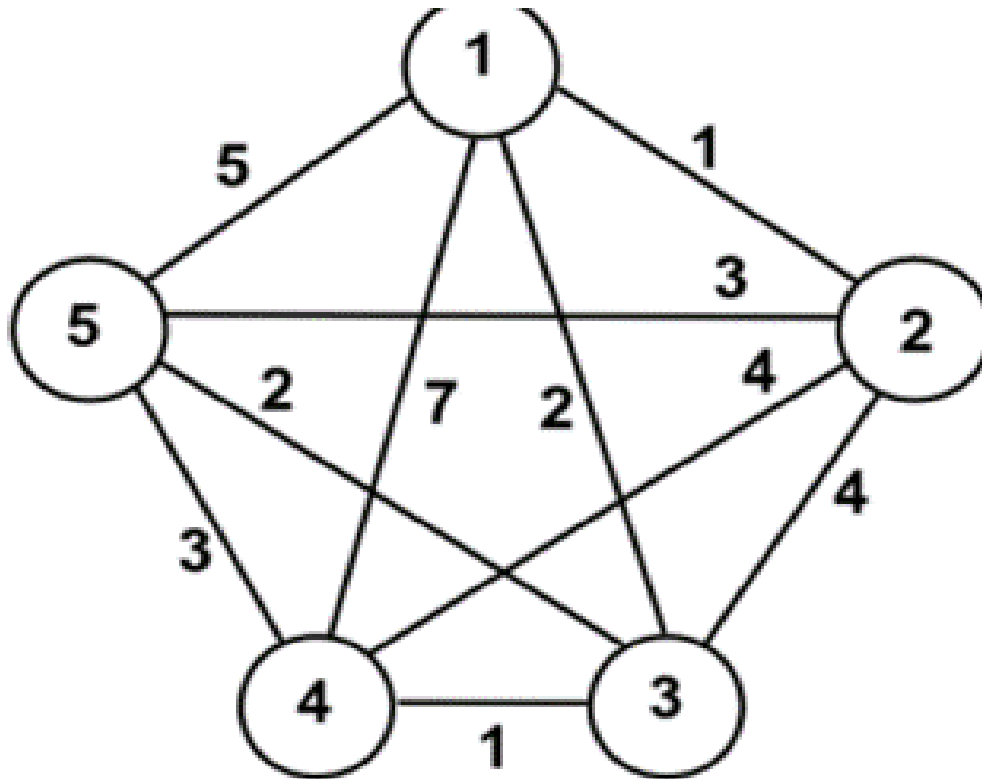


#nhận xét: input, output, vẽ cây khung nhỏ nhất?

#Thực hành 02:

Kruskal's algorithm

Minimum Spanning Tree of a given connected,
đồ thị vô hướng, có trọng số



#Out:

Edges in the constructed MST

0 -- 1 == 1

2 -- 3 == 1

2 -- 0 == 2

2 -- 4 == 2

Minimum Spanning Tree 6

#nhận xét: input, output và vẽ cây khung nhỏ nhất?

#Thực hành 03:

Kruskal's algorithm

Minimum Spanning Tree of a given connected,
đồ thị vô hướng, có trọng số

	A	B	C	D	E	F
A	0	2	3	0	0	0
B	2	0	0	5	2	0
C	3	0	0	0	5	0
D	0	5	0	0	1	2
E	0	2	5	1	0	4
F	0	0	0	2	4	0

#out:

Edges in the constructed MST

3 -- 4 == 1

0 -- 1 == 2

1 -- 4 == 2

3 -- 5 == 2

0 -- 2 == 3

Minimum Spanning Tree 10

#nhận xét: input, output và vẽ đồ thị, cây khung nhỏ nhất?

TH 04 - Heuristic A* Algorithm

```
from collections import deque
class Graph:
    def __init__(self, adjac_lis):
        self.adjac_lis = adjac_lis

    def get_neighbors(self, v):
        return self.adjac_lis[v]

    def h(self, n):
        H = {
            'A': 1,
            'B': 1,
            'C': 1,
            'D': 1
        }
        return H[n]

    def a_star_algorithm(self, start, stop):

        open_lst = set([start])
        closed_lst = set([])

        poo = {}
        poo[start] = 0

        par = {}
        par[start] = start
```

TH 04 - Heuristic A* Algorithm

```
while len(open_lst) > 0:
    n = None
    for v in open_lst:
        if n == None or poo[v] + self.h(v) < poo[n] + self.h(n):
            n = v;

    if n == None:
        print('Path does not exist!')
        return None

    if n == stop:
        reconst_path = []
        while par[n] != n:
            reconst_path.append(n)
            n = par[n]

        reconst_path.append(start)

        reconst_path.reverse()

    print('Path found: {}'.format(reconst_path))
    return reconst_path
```

TH 04 - Heuristic A* Algorithm

```
    for (m, weight) in self.get_neighbors(n):
        if m not in open_lst and m not in closed_lst:
            open_lst.add(m)
            par[m] = n
            poo[m] = poo[n] + weight

        else:
            if poo[m] > poo[n] + weight:
                poo[m] = poo[n] + weight
                par[m] = n

            if m in closed_lst:
                closed_lst.remove(m)
                open_lst.add(m)

    open_lst.remove(n)
    closed_lst.add(n)

    print('Path does not exist!')
    return None
```

TH 04 - Heuristic A* Algorithm

```
#INPUT:
if __name__ == '__main__':
    h = {
        'A': [('B', 1), ('C', 1), ('D', 4)],
        'B': [('D', 5)],
        'C': [('D', 2)],
    }
    g = Graph(h)
    g.a_star_algorithm('A', 'D')
#OUTPUT:
Path found: ['A', 'C', 'D']
```

#nhận xét: input, output và vẽ đồ thị?

TH 05 - Heuristic A* Algorithm

```
from collections import deque
```

```
class Graph:
```

```
    def __init__(self, adjac_lis):  
        self.adjac_lis = adjac_lis
```

```
    def get_neighbors(self, v):  
        return self.adjac_lis[v]
```

```
    def h(self, n):
```

```
        H = {
```

```
            'A': [('B', 1), ('C', 3), ('D', 7)],
```

```
            'B': [('D', 5)],
```

```
            'C': [('D', 12)],
```

```
        }
```

```
        return H[n]
```

```
    def a_star_algorithm(self, start, stop):
```

```
        open_lst = set([start])
```

```
        closed_lst = set([])
```

```
        poo = {}
```

```
        poo[start] = 0
```

```
        par = {}
```

```
        par[start] = start
```

TH 05 - Heuristic A* Algorithm

```
while len(open_lst) > 0:
    n = None

    for v in open_lst:
        if n == None or poo[v] + self.h(v) < poo[n] + self.h(n):
            n = v;

    if n == None:
        print('Path does not exist!')
        return None

    if n == stop:
        reconst_path = []

        while par[n] != n:
            reconst_path.append(n)
            n = par[n]

        reconst_path.append(start)

        reconst_path.reverse()

        print('Path found: {}'.format(reconst_path))
        return reconst_path
```

TH 05 - Heuristic A* Algorithm

```
for (m, weight) in self.get_neighbors(n):

    if m not in open_lst and m not in closed_lst:
        open_lst.add(m)
        par[m] = n
        poo[m] = poo[n] + weight

    else:
        if poo[m] > poo[n] + weight:
            poo[m] = poo[n] + weight
            par[m] = n

        if m in closed_lst:
            closed_lst.remove(m)
            open_lst.add(m)

open_lst.remove(n)
closed_lst.add(n)

print('Path does not exist!')
return None
```

TH 05 - Heuristic A* Algorithm

```
#INPUT:
if __name__ == '__main__':
    adjac_lis = {
        'A': [('B', 2), ('C', 4)],
        'B': [('A', 2), ('C', 1), ('D', 5)],
        'C': [('A', 4), ('B', 1), ('D', 3)],
        'D': [('B', 5), ('C', 3)]
    }
    g = Graph(adjac_lis)
    start_node = str(input("Enter the start node:"))
    stop_node = str(input("Enter the stop node:"))

    g.a_star_algorithm(start_node, stop_node)
#OUTPUT:
Enter the start node:A
Enter the stop node:D
Path found: ['A', 'B', 'C', 'D']
```

#nhận xét: input, output và vẽ đồ thị?

#Thực hành 06:

#8 puzzle problem

Giải pháp câu đố, vấn đề câu đố 8 được thảo luận như sau:

+Cho một bảng 3×3 có 8 ô (mỗi ô có một số từ 1 đến 8) và một ô trống.

+Mục tiêu là đặt các số trên các ô để khớp với cấu hình cuối cùng bằng cách sử dụng khoảng trống.

+Có thể trượt bốn ô liền kề (trái, phải, trên và dưới) vào khoảng trống

Initial
configuration

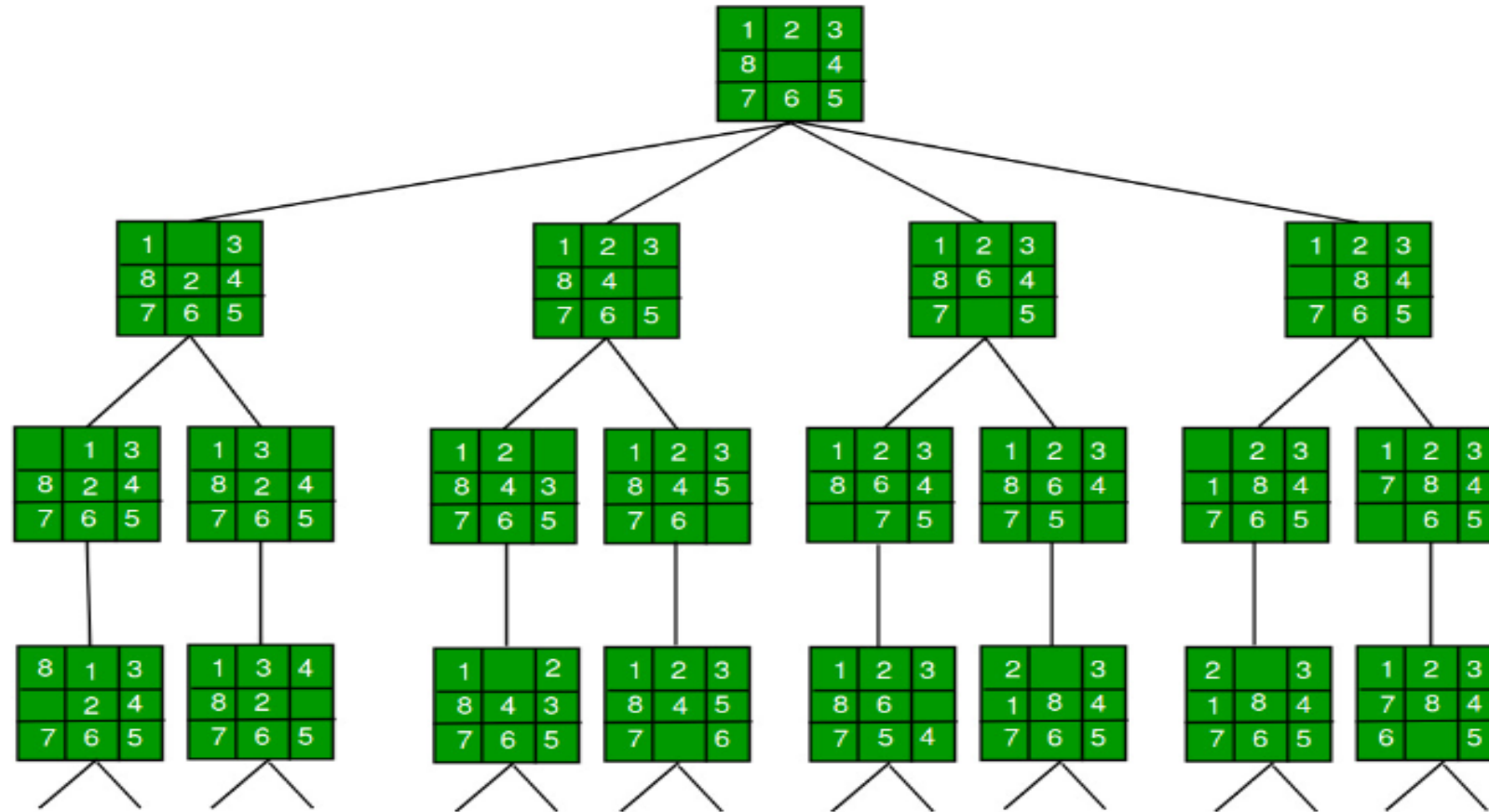
1	2	3
5	6	
7	8	4

Final
configuration

1	2	3
5	8	6
	7	4

#Thực hành 06:

#8 puzzle problem



+Sơ đồ cho thấy con đường mà thuật toán trên theo sau để đạt được cấu hình cuối cùng từ cấu hình ban đầu đã cho của 8-Puzzle. Các nút có giá trị nhỏ nhất của hàm chi phí mới được mở rộng

#Thực hành 06:

#8 puzzle problem

```
import copy
from heapq import heappush, heappop
n = 3
# bottom, left, top, right
row = [ 1, 0, -1, 0 ]
col = [ 0, -1, 0, 1 ]

#lớp Queue ưu tiên
class priorityQueue:

    def __init__(self):
        self.heap = []
        # chèn mới 'k'
    def push(self, k):
        heappush(self.heap, k)

    # phương thức loại bỏ ptu nhỏ nhất
    def pop(self):
        return heappop(self.heap)

    # phương thức biết Queue trống hay ko
    def empty(self):
        if not self.heap:
            return True
        else:
            return False
```

#Thực hành 06:

#8 puzzle problem

```
# cấu trúc nút
class node:

    def __init__(self, parent, mat, empty_tile_pos,
                  cost, level):

        # lưu trữ nút cha
        self.parent = parent
        # lưu trữ ma trận
        self.mat = mat
        # lưu trữ vị trí tại đó

        self.empty_tile_pos = empty_tile_pos

        self.cost = cost

        self.level = level

    def __lt__(self, nxt):
        return self.cost < nxt.cost

def calculateCost(mat, final) -> int:
    count = 0
    for i in range(n):
        for j in range(n):
            if ((mat[i][j]) and
                (mat[i][j] != final[i][j])):
                count += 1

    return count
```

#Thực hành 06:

#8 puzzle problem

```
def newNode(mat, empty_tile_pos, new_empty_tile_pos,
            level, parent, final) -> node:

    new_mat = copy.deepcopy(mat)

    x1 = empty_tile_pos[0]
    y1 = empty_tile_pos[1]
    x2 = new_empty_tile_pos[0]
    y2 = new_empty_tile_pos[1]
    new_mat[x1][y1], new_mat[x2][y2] = new_mat[x2][y2], new_mat[x1][y1]

    cost = calculateCost(new_mat, final)

    new_node = node(parent, new_mat, new_empty_tile_pos,
                    cost, level)
    return new_node

# ma trận N x N
def printMatrix(mat):

    for i in range(n):
        for j in range(n):
            print("%d " % (mat[i][j]), end = " ")

    print()
```

#Thực hành 06:

#8 puzzle problem

```
def isSafe(x, y):  
  
    return x >= 0 and x < n and y >= 0 and y < n  
# Print đường dẫn nút - nút  
def printPath(root):  
    if root == None:  
        return  
    printPath(root.parent)  
    printMatrix(root.mat)  
    print()  
def solve(initial, empty_tile_pos, final):  
  
    pq = priorityQueue()  
  
    # tạo nút root  
    cost = calculateCost(initial, final)  
    root = node(None, initial,  
                empty_tile_pos, cost, 0)  
    # add nút gốc vào danh sách các nút trực tiếp  
    pq.push(root)
```

#Thực hành 06:

#8 puzzle problem

```
while not pq.empty():

    minimum = pq.pop()
    if minimum.cost == 0:

        printPath(minimum)
        return
    for i in range(4):
        new_tile_pos = [
            minimum.empty_tile_pos[0] + row[i],
            minimum.empty_tile_pos[1] + col[i], ]

        if isSafe(new_tile_pos[0], new_tile_pos[1]):

            # tạo nút con
            child = newNode(minimum.mat,
                            minimum.empty_tile_pos,
                            new_tile_pos,
                            minimum.level + 1,
                            minimum, final,)

            pq.push(child)
```

#Thực hành 06:

#8 puzzle problem

```
# Giá trị 0 được sử dụng cho không gian trống
initial = [ [ 1, 2, 3 ],
            [ 5, 6, 0 ],
            [ 7, 8, 4 ] ]

final = [ [ 1, 2, 3 ],
          [ 5, 8, 6 ],
          [ 0, 7, 4 ] ]

empty_tile_pos = [ 1, 2 ]
solve(initial, empty_tile_pos, final)
```

#nhận xét: input, output?



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

